Chapter 01

Introduction to Python

[*] Level	[#] Topics	[t] Est. Time	[py] Python
Absolute Beginner	5 Core Concepts	2 - 3 Hours	3.x

Table of Contents

1.1 What is Python?

Python is a **high-level, interpreted, general-purpose programming language** created by **Guido van Rossum** and first released in **1991**. The name comes not from the snake, but from the British comedy series *Monty Python's Flying Circus* - which tells you a lot about the language's philosophy: it is meant to be fun and readable.

> **[i] NOTE**
> High-level language = a language closer to human English than to 0s and 1s.
> The computer cannot run English directly, so Python translates your words
> into machine code automatically. You focus on the idea, Python handles the rest.

Key Characteristics

- **Readable syntax** - Python code looks almost like plain English. Compare C++ vs Python to print Hello World - Python wins every time.
- **Interpreted** - You don't need to 'compile' your code before running it. Python runs it line-by-line, which makes debugging much easier.
- **Dynamically typed** - You never need to declare the type of a variable. Python figures it out on its own.
- **Multi-paradigm** - Python supports procedural, object-oriented, and functional programming. You pick the style that suits the problem.
- **Huge standard library** - Batteries included - Python ships with modules for files, dates, math, networking, and hundreds more.
- **Cross-platform** - The same Python file runs on Windows, macOS, and Linux with zero changes.

Python in the Real World

Python is used in virtually every field of computing today:

Domain	What Python Does	Famous Examples
Web Development	Build server-side websites & APIs	Django, Flask, FastAPI
Data Science	Analyse large datasets, build charts	Pandas, Matplotlib
Machine Learning	Train models that learn from data	TensorFlow, PyTorch
Automation	Script repetitive tasks, bots	Selenium, PyAutoGUI
Cybersecurity	Penetration testing, scripts	Scapy, Impacket
Finance	Algorithmic trading, risk analysis	Quantlib, yfinance
Game Dev	Prototype games, scripting engines	Pygame, Ren'Py
Scientific Research	Simulations, data processing	NumPy, SciPy

1.2 Why Learn Python?

You could learn any language first. Here is why Python is the best choice for a beginner who also wants to be employable:

[+] TIP
#1 Most Popular
Python has ranked #1 on the TIOBE Index and Stack Overflow Developer Survey multiple times. More popularity means more tutorials, more jobs, and more community help when you get stuck.

[+] TIP
Easiest Syntax
A Python Hello World is one line. In Java it is 7 lines of boilerplate. This lets you focus on learning programming concepts, not language ceremony.

[+] TIP
Fast to Prototype
You can build a working idea in Python in hours. Companies love this for quick experiments and MVPs (Minimum Viable Products).

[+] TIP
High Salaries
Python developers are among the highest-paid in tech. AI/ML engineers (almost all Python) command premium salaries globally.

[+] TIP
Versatile
You learn Python once, then apply it to web, data, automation, and AI. You are never locked into one specialty.

Proof: Hello World Comparison

Java - Hello World (7 lines of ceremony)

```
# Java requires a class, a method, and ceremonial boilerplate:
```

Python - Hello World (1 line, done)

```
print("Hello, World!")
```

1.3 How Python Works - The Interpreter

This is something most beginner tutorials skip - but it is *crucial* to understand so you are never confused about what is happening behind the scenes.

Compiled vs Interpreted Languages

Programming languages fall into two broad categories based on how they are converted into machine instructions:

Type	Behaviour
Compiled (C, C++, Rust)	Entire source code -> machine code BEFORE running. Fast execution. Need to recompile for each OS.
Interpreted (Python, JavaScript)	Source code -> executed line-by-line AT RUNTIME. Slower but portable and interactive.

What Actually Happens When You Run a .py File

The full pipeline is more interesting than 'just interpreted'. Here is what Python actually does:

Step 1 - Lexing	Python reads your .py file character by character and breaks it into tokens (words, symbols, numbers).
Step 2 - Parsing	Tokens are assembled into an Abstract Syntax Tree (AST) - a tree structure representing your program's logic.
Step 3 - Compilation	The AST is compiled into Python Bytecode (.pyc files stored in __pycache__). This is NOT machine code - it is an intermediate format.
Step 4 - CPython VM	The Python Virtual Machine (CPython) reads the bytecode and executes it. CPython is itself a C program.

[?] THINK ABOUT IT

This is why Python is called 'interpreted' - but it actually compiles to bytecode first!
The .pyc files in __pycache__ are bytecode. They speed up future runs because
Python skips steps 1-3 if the source hasn't changed.

CPython vs Other Python Implementations

When people say 'Python', they usually mean CPython - the reference implementation written in C. But there are others:

Implementation	Description
CPython	The standard. What you install from python.org.

Implementation	Description
PyPy	Faster CPython with JIT compilation. Great for CPU-heavy code.
Jython	Python running on the Java Virtual Machine.
IronPython	Python running on .NET/C#.
MicroPython	Tiny Python for microcontrollers (Arduino, Raspberry Pi Pico).

1.4 Installing Python & Setting Up VS Code

Installing Python

Installation Steps

```
# Step 1: Go to https://www.python.org/downloads/
# Step 2: Download the latest Python 3.x installer
# Step 3: Run the installer

# CRITICAL: On Windows, check the box that says:
#   'Add Python to PATH'
# If you miss this, Python won't work from the terminal!

# Step 4: Verify installation in terminal:
python --version
# Expected output: Python 3.12.x  (or whatever latest version)

python3 --version   # On macOS/Linux, use python3
```

[i] NOTE

PATH is a list of folders your operating system searches when you type a command. Adding Python to PATH means you can type 'python' from any folder in your terminal and the OS will find the Python program automatically.

Setting Up VS Code

	Download VS Code	Go to https://code.visualstudio.com and install it.
2	**Install Python Extension**	Open VS Code -> Extensions (Ctrl+Shift+X) -> search 'Python' -> Install (by Microsoft).
	Select Python Interpreter	Press Ctrl+Shift+P -> type 'Python: Select Interpreter' -> choose your Python 3.x version.
4	**Create a .py file**	File -> New File -> Save as hello.py. VS Code auto-activates Python features.

	Run your file	Click the > button top-right, OR press Ctrl+F5, OR right-click -> 'Run Python File'.

Recommended VS Code Extensions for Python

Extension	Author	Why It Helps
Python	Microsoft	Core Python support, IntelliSense, debugging
Pylance	Microsoft	Faster, smarter code analysis and type hints
Indent-Rainbow	oderwat	Colours indentation levels - great for beginners
autopep8 / Black	Various	Auto-formats your code to PEP 8 standards
GitLens	GitKraken	Shows git blame inline - useful from day one

1.5 Your First Python Program

Let us write, run, and fully understand the simplest Python program. We will then progressively make it more interesting.

hello.py - Your First Program

```
# This is our first Python file
# File: hello.py

print("Hello, World!")
```

Output:

Terminal Output

```
Hello, World!
```

Breaking It Down - Every Character Matters

`print`	A built-in function. A function is a named block of code that does a specific job. print's job is to display output.
`( )`	Parentheses call the function. Everything inside the () is passed to the function as input.
`"Hello, World!"`	A string (text data). Strings must be wrapped in quotes - single or double. The computer needs to distinguish text from code.
`"`	Opening and closing quotes mark where the string starts and ends. They must match - don't open with " and close with '.

Level Up: A More Interesting First Program

greet_user.py

```
# File: greet_user.py

# Ask the user for their name
name = input('What is your name? ')

# Greet them using their name
print('Hello,', name, '!')
print('Welcome to Python!')
```

Sample Run

```
What is your name? Aryan
Hello, Aryan !
Welcome to Python!
```

> **[i] NOTE**
> input() pauses the program and waits for the user to type something and press Enter. Whatever they type is returned as a string. We store it in a variable called 'name'. Variables are covered in Chapter 02.

1.6 The print() Function - Deep Dive

print() is the very first function you learn, but most beginners only scratch its surface. Here is everything it can do.

Full Signature

print() - Full Function Signature

```
print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)
```

This looks scary at first - let us break each parameter down:

Parameter	Meaning
*objects	One or more values to print. The * means you can pass as many as you like.
sep=' '	Separator between multiple values. Default is a single space.
end='\n'	What to print at the very end. Default is a newline (\n), so each print() starts on a new line.
file=sys.stdout	Where to print. Default is the terminal. You can redirect to a file.

Parameter	Meaning
flush=False	Whether to force-flush the output buffer. Rarely needed as a beginner.

Examples - From Basic to Advanced

1. Printing multiple values:

Multiple values

```
print('Python', 'is', 'awesome')
# Output: Python is awesome

# The spaces between words come from sep=' ' (the default separator)
```

2. Changing the separator:

Custom separators

```
print('2024', '06', '11', sep='-')
# Output: 2024-06-11

print('a', 'b', 'c', sep=', ')
# Output: a, b, c

print('one', 'two', 'three', sep='')
# Output: onetwothree   (no separator at all)
```

3. Changing the end character (the tricky one):

end parameter

```
# Default behaviour: each print ends with a newline
print('Line 1')
print('Line 2')
# Output:
# Line 1
# Line 2

# Custom end: stay on the same line
print('Hello', end=' ')
print('World')
# Output: Hello World    <- on a SINGLE line!

# Practical use: print a loading bar on one line
print('Loading', end='')
print('...', end='')
print(' Done!')
# Output: Loading... Done!
```

4. Printing nothing (blank line):

Blank line with print()

```
print('Section 1')
print()          # prints an empty line (just the default newline)
print('Section 2')
# Output:
# Section 1
#
# Section 2
```

5. Printing to a file:

Printing to a file

```
# This writes to a file instead of the terminal
with open('output.txt', 'w') as f:
    print('This goes to a file!', file=f)

# Don't worry about the 'with open' syntax yet - that is Chapter 13.
# Just know that print() can do this.
```

[?] THINK ABOUT IT

The \n escape sequence stands for 'newline character'. It is invisible - it just moves the cursor to the next line. print() automatically adds \n at the end of everything it prints. That is why each print() appears on its own line by default. You will see many escape sequences like this.

Escape Sequences in Strings

An escape sequence starts with a backslash (\) and tells Python to treat the following character specially:

Sequence	Meaning	Example
\n	Newline - moves to the next line	print("Hello\nWorld") -> Hello (newline) World
\t	Tab - inserts a horizontal tab	print("Name:\tAryan") -> Name: (tab) Aryan
\\	Literal backslash - prints one \	print("C:\\Users") -> C:\Users
\"	Literal double quote inside a string	print("He said \"hi\"") -> He said "hi"
\'	Literal single quote inside a string	print('It\'s fine') -> It's fine
\r	Carriage return - goes to line start	Rarely used in modern code

1.7 Comments - Why & How

A comment is text in your code that Python completely ignores. Comments are for **human readers** - your future self, your teammates, your teacher. Writing good comments is a professional skill.

Single-line Comments - The # Symbol

Single-line comments

```
# This is a comment. Python ignores this entire line.

print('Hello')  # This is an inline comment. Code runs; comment is ignored.

# You can comment out code to temporarily disable it:
# print('This line will NOT run')
print('This line WILL run')
```

Multi-line Comments - Triple Quotes

Python has no dedicated multi-line comment syntax. The convention is to use triple-quoted strings:

Multi-line comments

```
"""
This is a multi-line
comment. Python sees it as a string
but because it's not assigned to anything,
it is effectively ignored.
"""

# Alternatively:
# Line 1 of comment
# Line 2 of comment
# Line 3 of comment
```

Docstrings - Comments That Serve Double Duty

When a triple-quoted string appears as the very first statement inside a function, class, or module, it becomes a **docstring**. Python stores it and makes it accessible with `help()`. You will use these a lot in Chapter 11.

Docstrings

```
def greet(name):
    """
    Greets a user by name.

    Args:
        name (str): The name of the user.

    Returns:
        None
    """
    print('Hello,', name)

help(greet)  # prints the docstring above!
```

Good vs Bad Comments

Comment quality

```
# BAD comment (just repeats the code - adds zero value):
x = 5  # sets x to 5

# GOOD comment (explains the WHY, not the WHAT):
retry_limit = 5  # max attempts before we lock the account

# BAD: outdated comment (code changed, comment didn't)
# multiply by 2 to convert feet to yards
result = value * 3  # BUG: someone changed the multiply but not the comment!

# GOOD: self-documenting code (sometimes the best comment is no comment):
FEET_TO_YARDS = 3
result = value * FEET_TO_YARDS
```

[+] TIP

The best code tells its own story through clear variable/function names.
Use comments to explain complex logic, edge cases, or the reasoning
behind non-obvious decisions - not to narrate what the code obviously does.

1.8 Python Syntax Rules

Syntax is the set of rules that define how Python code must be written. Break these rules and Python throws an error before your code even runs. Let us cover every rule a beginner must know.

Rule 1 - Indentation Is Mandatory

This is the most important Python syntax rule and the one that trips up people coming from other languages. In Python, **indentation (whitespace at the start of a line) defines code blocks**. This is not optional style - it is enforced by the language.

Indentation

```
# CORRECT: the code inside the if is indented by 4 spaces
if True:
    print('This is inside the if block')
    print('This is also inside')
print('This is OUTSIDE the if block')

# WRONG: missing indentation - IndentationError!
if True:
print('IndentationError: expected an indented block')
```

[!] IMPORTANT
The Python convention (PEP 8) is to use 4 SPACES for each indentation level. Never mix tabs and spaces - it causes subtle, hard-to-find bugs. Configure VS Code to 'insert spaces' when you press Tab (it does this by default for Python files).

Rule 2 - Case Sensitivity

Case sensitivity

```
# Python is CASE SENSITIVE - these are three different things:
name = 'Aryan'   # variable 'name'
Name = 'Riya'    # different variable 'Name'
NAME = 'Admin'   # different variable 'NAME'

print(name)   # Aryan
print(Name)   # Riya
print(NAME)   # Admin

# This causes a NameError:
Print('Hello')   # Error! Built-in is 'print', not 'Print'
```

Rule 3 - One Statement Per Line (Usually)

Statements

```
# Standard: one statement per line
x = 10
y = 20
z = x + y

# Multiple statements on one line with ; - WORKS but is BAD STYLE
x = 10; y = 20; z = x + y  # Don't do this - hard to read

# Long lines can be split with \
result = 100 + 200 + 300 + \
         400 + 500

# Or use parentheses for implicit line continuation (preferred):
result = (100 + 200 + 300 +
          400 + 500)
```

Rule 4 - Naming Rules (Identifiers)

An identifier is any name you give to a variable, function, or class. Python has strict rules:

Rule	Detail
Can start with	A letter (a-z, A-Z) or an underscore _
Can contain	Letters, digits (0-9), underscores
Cannot start with	A digit - 1name is invalid
Cannot contain	Spaces or special chars: @, -, ., etc.
Cannot be	A Python keyword: if, for, while, print, etc.

Identifier examples

```
# VALID identifiers:
name = 'Aryan'
user_age = 20
_private_var = 42
camelCase = 'some text'
MyClass = True

# INVALID identifiers - these will cause SyntaxError:
# 1name = 'bad'       starts with digit
# my-name = 'bad'     hyphen not allowed
# for = 5             'for' is a keyword
# user age = 20       space not allowed
```

Python Naming Conventions (PEP 8)

The rules above are enforced by Python. The following are conventions - rules that the community follows so all Python code looks consistent:

For	Convention	Example
variables & functions	snake_case	user_name, get_total()
constants	UPPER_SNAKE_CASE	MAX_RETRIES, PI
classes	PascalCase / CapWords	BankAccount, UserProfile
private (internal)	_leading_underscore	_helper_func()
'dunder' (special)	__double_underscore__	__init__, __str__

Python Keywords - Never Use These as Names

These 35 words are reserved by Python. You cannot use them as variable names:

`False`	`None`	`True`	`and`	`as`
`assert`	`async`	`await`	`break`	`class`
`continue`	`def`	`del`	`elif`	`else`
`except`	`finally`	`for`	`from`	`global`
`if`	`import`	`in`	`is`	`lambda`
`nonlocal`	`not`	`or`	`pass`	`raise`
`return`	`try`	`while`	`with`	`yield`

1.9 How Python Reads Your Code (Behind the Scenes)

Understanding the order in which Python processes your code prevents an entire category of confusing bugs.

Execution Order - Top to Bottom, Left to Right

Top-to-bottom execution

```
# Python ALWAYS executes line 1 before line 2, line 2 before line 3...

print('Step 1')    # runs first
print('Step 2')    # runs second
print('Step 3')    # runs third

# Output:
# Step 1
# Step 2
# Step 3
```

The Two Phases: Compilation vs Execution

Two phases

```
# Phase 1 - Compilation (before anything runs):
# Python scans the ENTIRE file for syntax errors.
# If it finds one, NOTHING runs - not even the lines above the error.

print('I will never run')  # This is above the error
x = 10 +                   # SyntaxError: incomplete expression
print('Neither will I')    # This is below the error

# Phase 2 - Execution (line by line, only if Phase 1 passes):
# Each line is executed one at a time.
# A runtime error (like division by zero) only stops execution
# at THAT line - lines above it have already run.
```

REPL - Interactive Python Mode

You can run Python interactively - one line at a time. This is called the REPL (Read-Eval-Print Loop). It is great for quick experiments:

REPL - Interactive Mode

```
# Open terminal and type: python  (or python3)
# You will see the >>> prompt:

>>> 2 + 3
5
>>> print('Testing')
Testing
>>> name = 'Aryan'
>>> name
'Aryan'
>>> exit()   # to quit the REPL
```

[?] THINK ABOUT IT
In the REPL, if you type an expression (like 2 + 3 or name), Python automatically prints its value. In a .py file, you must use print() explicitly - just writing '2 + 3' in a file does nothing visible.

1.10 Common Beginner Mistakes (and How to Fix Them)

SyntaxError: missing brackets or quotes

```
print "Hello"        # Python 2 syntax - broken in Python 3
print("Hello")       # CORRECT
```

[+] TIP
Fix: Always use parentheses with print().

IndentationError: unexpected indent

```
print("Hello")
    print("World")   # Random indentation causes IndentationError
```

[+] TIP
Fix: Only indent code that belongs inside a block (if, for, def, etc.).

NameError: name is not defined

```
print(massage)    # NameError: typo - should be "message"
message = "Hello"
print(message)    # CORRECT (and define before using!)
```

[+] TIP
Fix: Define variables before using them. Check for typos.

TypeError: mixing data types carelessly

```
age = 20
print("I am " + age)   # TypeError: can only concatenate str to str
print("I am " + str(age))  # CORRECT: convert to string first
```

[+] TIP
Fix: When combining strings and numbers, convert numbers with str().

Using = instead of == for comparison

```
if x = 5:    # SyntaxError! = is assignment, not comparison
if x == 5:   # CORRECT: == checks equality
```

[+] TIP
Fix: = assigns a value. == compares two values. Common mistake!

1.11 Chapter 01 - Quick Reference Card

Syntax / Command	Description
print(value)	Display output to the terminal
print(a, b, c)	Print multiple values separated by spaces
print(a, b, sep="-")	Print with custom separator
print(a, end="")	Print without newline at end
input('prompt')	Read user input as a string
# comment	Single-line comment (ignored by Python)
""" text """	Multi-line comment / docstring
\ at line end	Continue a long statement on the next line
(expression)	Parentheses also allow implicit line continuation
python file.py	Run a Python file from terminal
python	Enter the REPL (interactive mode)
exit()	Exit the REPL

What's Coming Next

Chapter	Topic
Chapter 02	Variables & Data Types - store and work with data
Chapter 03	Operators - perform calculations and comparisons
Chapter 04	Strings - master Python's most used data type
Chapter 05	Lists - store collections of data

[+] TIP

You have covered the foundations of Python - what it is, how it works internally, how to install it, and the core syntax rules. Every chapter from here builds on this foundation. Before moving to Chapter 02, complete the Chapter 01 Practice Set - typing code yourself is the ONLY way to build muscle memory.